GLOWLOGICS

CYBERSECURITY AND ETHICAL HACKING

INTERNSHIP PROJECT REPORT

Project Title

Password Strength Analyzer

PROJECT SUBMITTED BY

SHIVAANI SHREE SR

Email: shivaani.050@gmail.com

Submitted to: GlowLogics

CERTIFICATE / SUBMISSION NOTE

This report presents a beginner-level cybersecurity project titled **“Password Strength Analyzer.”** It demonstrates how password length and character variety can be checked locally and how users can receive practical suggestions for creating stronger passwords.

Note: This is a project report prepared for learning and submission purposes. It is not an official internship certificate, company approval, or proof of completion. Add any required company-provided certificate or approval separately.

ABSTRACT

Password security is an important part of protecting online accounts. This project implements a simple Python program that evaluates a password using its length and the presence of lowercase letters, uppercase letters, digits, and special characters. It gives a basic strength label and improvement tips. The analysis is performed locally; the program does not transmit or save the entered password.

1. INTRODUCTION

Weak or reused passwords can make accounts easier to compromise. A password strength checker helps users understand common password rules and encourages better choices. This project is designed as an educational demonstration of string handling, conditional logic, input validation, and regular expressions in Python.

2. OBJECTIVES

• Check password length and character variety.
• Classify a password as Weak, Moderate, or Strong using simple rules.
• Give useful feedback for improving a weak password.
• Demonstrate basic scripting and input validation.
• Promote safe password habits without storing the entered password.

3. TOOLS AND TECHNOLOGIES

• Programming language: Python 3
• Standard libraries: re (regular expressions) and getpass (hidden terminal input)
• Platform: Any system with Python 3 installed
• No external packages or internet connection are required.

4. SYSTEM DESIGN

Input: A password typed by the user in the terminal.
Processing: The program checks length and character categories.
Output: A strength label and suggestions for improvement.
Privacy: The entered password is checked in memory only and is not written to a file or sent over a network.

5. ALGORITHM

1. Start the program.
2. Ask the user to enter a password using hidden terminal input.
3. Check whether the password has at least 8 characters and whether it contains lowercase letters, uppercase letters, digits, and special characters.
4. Count the satisfied character categories.
5. Apply the simple scoring rules in the program.

6. Display the strength level and suggestions.
7. End the program.

6. IMPLEMENTATION (PYTHON 3)

Save the following code as **password_analyzer.py** and run it from a terminal using **python password_analyzer.py**. The input is hidden while typing when the terminal supports getpass.

```
import re
from getpass import getpass

def analyze_password(password):
    score = 0
    tips = []

    if len(password) >= 12:
        score += 2
    elif len(password) >= 8:
        score += 1
    else:
        tips.append("Use at least 8 characters; 12 or more is better.")

    if re.search(r"[a-z]", password):
        score += 1
    else:
        tips.append("Add lowercase letters.")

    if re.search(r"[A-Z]", password):
        score += 1
    else:
        tips.append("Add uppercase letters.")

    if re.search(r"\d", password):
        score += 1
    else:
        tips.append("Add numbers.")

    if re.search(r"[^A-Za-z0-9]", password):
        score += 1
    else:
        tips.append("Add a special character, such as ! or #.")

    if score >= 6 and len(password) >= 12:
        strength = "Strong (basic rule-based estimate)"
    elif score >= 4:
        strength = "Moderate (basic rule-based estimate)"
    else:
        strength = "Weak (basic rule-based estimate)"

    return strength, tips

def main():
    print("=== Password Strength Analyzer ===")
    password = getpass("Enter a password (input is hidden): ")

    strength, tips = analyze_password(password)
    print("\\nResult:", strength)

    if tips:
        print("Suggestions:")
        for tip in tips:
            print("-", tip)
    else:
        print("Good job! Your password meets all checks.")

    print("\\nPrivacy note: This program does not save or transmit your password.")

if __name__ == "__main__":
    main()
```

7. SAMPLE OUTPUT

```
=== Password Strength Analyzer ===
Enter a password (input is hidden): [hidden]

Result: Strong (basic rule-based estimate)
Good job! Your password meets all checks.
```

```
Privacy note: This program does not save or transmit your password.
```

The exact result depends on the password entered. The program intentionally does not show the password back on screen.

8. TEST CASES

Test input pattern	Expected result	Reason
Short word only	Weak	Too short and lacks character variety
Longer word with mixed case	Moderate	Length and case checks pass, but other categories may be missing
12+ characters with mixed case, digits, and symbols	Strong	Meets the program's basic length and variety checks

9. LIMITATIONS

This is a beginner-friendly, rule-based checker, not a complete security audit. It does not estimate real-world cracking time, detect breached passwords, identify dictionary words, or check whether a password has been reused. A “Strong” result is not a guarantee of safety.

10. SECURITY RECOMMENDATIONS

• Use a unique password for every account.
• Prefer long passphrases or randomly generated passwords.
• Consider using a reputable password manager.
• Enable multi-factor authentication where available.
• Never share passwords or use real personal passwords for project demonstrations.

11. FUTURE ENHANCEMENTS

Possible improvements include a graphical user interface, a password entropy estimate, detection of common password patterns, and an optional local-only report. Any future feature should protect privacy and must not store plain-text passwords.

12. CONCLUSION

The Password Strength Analyzer demonstrates basic password security concepts using Python. It checks password length and character variety, provides improvement tips, and introduces input validation and regular expressions. The project can be extended with more advanced checks while keeping user privacy as a priority.

REFERENCES

1. Python documentation: https://docs.python.org/3/
2. Python regular expression documentation: https://docs.python.org/3/library/re.html
3. CISA guidance on strong passwords and multi-factor authentication: https://www.cisa.gov/secure-our-world

Submitted by: SHIVAANI SHREE SR
Email: shivaani.050@gmail.com
Organization: GlowLogics